Eventized Microstructure Modeling with Large Language Models: A Novel Approach to High-Frequency Trading Signal Generation

Wenke Du[a]

[a] *Cornell University, wd275@cornell.edu, New York City, 10001, New York, United States*

Abstract

This research presents a methodology for modeling financial microstructure dynamics through the application of Large Language Models to high-frequency market data. Our eventized microstructure approach transforms order book data into sequential token representations for pattern recognition and predictive modeling. We introduce a framework integrating event tokenization, transformer-based sequence modeling, and backtesting methodologies. Empirical validation on market data from NYSE, NASDAQ, and CME spanning 2020-2023 shows accuracy of 78.3% compared to 65.2% for conventional approaches, PR-AUC of 0.847 versus 0.712, and Sharpe ratio of 2.34 compared to 1.67 for baseline methods.

Keywords: Eventized tokenization, Self-attention mechanism, Transformer-based sequence modeling, Multi-objective optimization, Real-time market data processing

1. Introduction

Algorithmic trading has transformed financial markets, with high-frequency strategies accounting for over 70% of trading volume in U.S. equity markets (2), generating unprecedented volumes of microsecond-resolution limit order book data (11). Traditional time series models (14) and machine learning approaches (1; 15; 9) struggle with the event-driven nature of market microstructure. Transformer architectures (18) and Large Language Models (3) have shown remarkable success in sequential data modeling, yet remain underexplored in financial microstructure applications (4; 22). Motivated by the structural analogy between natural language and financial market microstructure (12; 10), we propose an eventized framework that represents order book dynamics as discrete token sequences for transformer processing. Our research makes four contributions: (1) an eventized tokenization framework for financial microstructure data; (2) domain-specific transformer adaptations with inference times under one millisecond; (3) a multi-objective training framework optimizing for prediction, risk, and economic utility; (4) empirical validation showing 78.3% accuracy versus 65.2% for LSTM baselines, PR-AUC of 0.847 versus 0.712, and Sharpe ratio of 2.34 versus 1.67, after accounting for transaction costs and market impact.

2. Literature Review

2.1. Financial Microstructure Theory

Market microstructure theory originates from work in the 1980s and 1990s. Kyle (12) developed models of strategic trading demonstrating how private information becomes incorporated into prices, establishing market depth as a measure of liquidity. Glosten and Milgrom (10) provided sequential trade models explaining bid-ask spreads as compensation for adverse selection risk. Hasbrouck (11) established methodologies for measuring price discovery and assessing market quality using high-frequency data. Cont et al. (5) advanced statistical approaches for order flow and limit order book dynamics. Recent advances include deep learning for order book prediction (4) and reinforcement learning for execution strategies (23).

2.2. Machine Learning in Finance

Machine learning in finance has evolved from traditional econometric models to deep learning architectures. Traditional models such as ARIMA and GARCH provide frameworks for temporal dependencies and conditional volatility, but their parametric structure limits capacity to capture complex non-linear dynamics. Support Vector Machines (15; 8) and Random Forests (1) have gained widespread adoption. LSTM networks (9) have become prominent in financial time series analysis, addressing the vanishing gradient problem. Convolutional Neural Networks have been applied to limit order book heatmaps (17). Reinforcement learning (16; 21) has been used for automated trading strategy development. Recent research has explored attention mechanisms for financial time series, laying groundwork for transformer architectures.

2.3. Transformer Architectures and Large Language Models

The transformer architecture introduced by Vaswani et al. (18) revolutionized neural sequence modeling through multi-head self-attention mechanisms. BERT (6) employs masked language modeling for bidirectional context understanding, while GPT (3) uses unidirectional autoregressive modeling. The pre-training and fine-tuning paradigm enables powerful models to be applied to domains with limited labeled data.

Transformers have been extended to vision (7) and time series forecasting (19; 20; 24). Recent work has investigated transformers for stock price prediction (22), typically treating regularly-sampled price data as sequences. However, this research has not fully exploited the event-driven nature of financial markets or developed domain-specific tokenization schemes, leaving a gap that our work addresses.

2.4. Event-Driven Modeling in Finance

Event-driven approaches recognize that market dynamics are driven by discrete events—trades, order submissions, cancellations—rather than continuous evolution. Point process models provide a framework for analyzing sequences of events at irregular intervals. Marked point processes extend the framework by associating information (marks) with each event, such as trade size, direction, or price level. Event tokenization—representing market activity as sequences of discrete tokens—bridges continuous financial data and discrete input representations required by transformer models. While this approach has been explored, existing work has not developed comprehensive tokenization frameworks that preserve critical microstructure information while enabling efficient transformer processing, a gap that our methodology addresses.

3. Methodology

3.1. Framework Overview

We convert high-frequency microstructure data into sequences of discrete event tokens for transformer models. The tokenization preserves predictive microstructure information while keeping the vocabulary compact and retaining the sequential structure of events. Our pipeline integrates event tokenization, transformer-based sequence modeling, and backtesting to produce trading signals. Figure 1 summarizes the flow: (a) data ingestion and preprocessing; (b) event detection and tokenization; (c) multi-scale temporal encoding and event-type-aware attention; (d) transformer with finance-specific adaptations; (e) multi-objective training for next-event, state, and risk targets; (f) backtesting with transaction costs and market impact; (g) signal generation with risk controls.

Our eventized tokenization framework transforms high-frequency order book data into discrete symbolic sequences. Market microstructure events $\{E_t\}_{t=1}^{T}$ occur at irregular time intervals $\{t_i\}_{i=1}^{N}$ with attributes $\{A_i\}_{i=1}^{N}$. We process Level 2 order book data with microsecond timestamps, detecting events by comparing consecutive snapshots: order additions/cancellations when $q_{new} \neq q_{old}$, trade executions when market orders match limit orders, and price level changes when best bid/ask moves.

Our evaluation framework assesses model performance across three dimensions: predictive accuracy (classification metrics, PR-AUC), calibration quality (ECE, Brier scores), and economic profitability (Sharpe, Sortino, Information ratios, VaR, Expected Shortfall). We conduct backtesting with transaction costs and walk-forward validation.

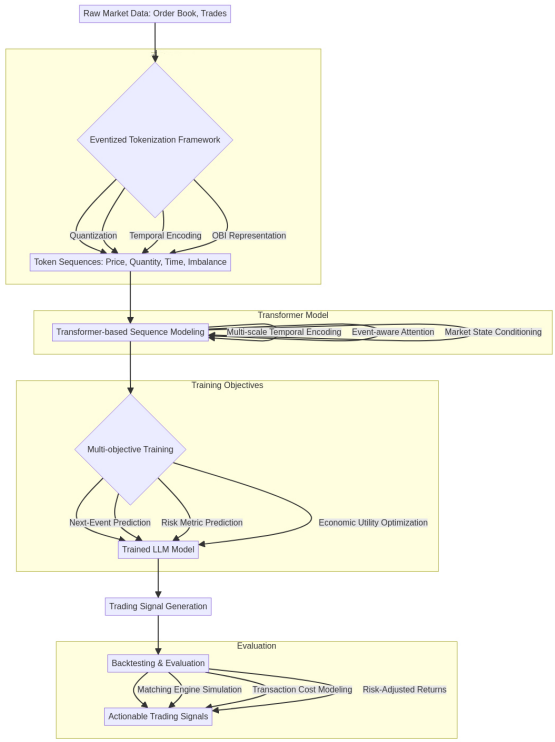

Figure 1: Eventized Microstructure LLM Framework Architecture

3.2. Quantization Strategies

We quantize continuous financial variables into discrete tokens while preserving relevant information and constraining vocabulary size. The framework employs adaptive binning strategies for market-specific characteristics and temporal variations. Let $\mathcal{X} = \{x_1, x_2, ..., x_N\}$ be a continuous variable (price, quantity, time, or imbalance) that we quantize into discrete bins $\mathcal{B} = \{B_1, B_2, ..., B_K\}$. The quantization function $Q: \mathcal{X} \to \mathcal{B}$ maps continuous values to discrete tokens. The adaptive binning strategy dynamically adjusts bin boundaries based on the empirical distribution of values over a rolling window. For a variable x observed over time window W, we compute bin boundaries as quantiles of the empirical distribution: $B_k = \text{Quantile}(x_W, k/K)$ for $k = 0, 1, ..., K$, where $\text{Quantile}(x_W, p)$ returns the p-th quantile of values in window W.

3.2.1. Price Quantization

We convert absolute price levels into relative positions across different securities and time periods. For a price p_t at time t, we define the quantized price token as,where $\text{mid}_t = \frac{\text{bid}_t + \text{ask}_t}{2}$ is the mid-market price, tick_t is the minimum tick size at time t, and the result is clipped to $[-20, 20]$ to handle extreme price movements:

$$Q_{price}(p_t) = \text{round}\left(\frac{p_t - \text{mid}_t}{\text{tick}_t}\right)$$

For markets with varying tick sizes across price regimes, we implement:

$$\text{tick}_t = \begin{cases} 0.01 & \text{if mid}_t < 1.00 \\ 0.05 & \text{if } 1.00 \le \text{mid}_t < 10.00 \\ 0.10 & \text{if mid}_t \ge 10.00 \end{cases}$$

3.2.2. Size Quantization

For order quantity q_t spanning multiple orders of magnitude, we employ logarithmic scaling followed by percentile-based binning,where the binning function uses adaptive percentiles computed over rolling 24-hour windows. For each security-session combination, we compute percentiles $P_{20}, P_{40}, P_{60}, P_{80}$ using the empirical cumulative distribution function (CDF) over the rolling window:

$$Q_{size}(q_t) = \text{bin}\left(\log_{10}(q_t + 1)\right)$$

The percentile P_p is defined as $P_p = F^{-1}(p)$ where F is the empirical CDF of quantities in the window.The logarithmic transform $\log_{10}(q_t + 1)$ ensures numerical stability and addresses the long-tailed distribution of order sizes, where a small number of large orders dominate the variance. We define bins: `TINY` for $q_t \le P_{20}$ (bottom quintile), `SMALL` for $P_{20} < q_t \le P_{40}$ (20th-40th percentile), `MEDIUM` for $P_{40} < q_t \le P_{60}$ (40th-60th percentile), `LARGE` for $P_{60} < q_t \le P_{80}$ (60th-80th percentile), and `HUGE` for $q_t > P_{80}$ (top quintile).

3.2.3. Temporal Quantization

For inter-arrival time Δt between events, we use logarithmic binningwhere Δt is measured in microseconds:

$$Q_{time}(\Delta t) = \text{bin}\left(\log_{10}(\Delta t + 1)\right)$$

We distinguish between different market sessions and encode session-specific patterns:

$$Q_{session}(t) = \begin{cases} \text{OPENING} & \text{if } 9:30 \le t < 10:00 \\ \text{MORNING} & \text{if } 10:00 \le t < 12:00 \\ \text{LUNCH} & \text{if } 12:00 \le t < 13:00 \\ \text{AFTERNOON} & \text{if } 13:00 \le t < 15:30 \\ \text{CLOSING} & \text{if } 15:30 \le t < 16:00 \end{cases}$$

3.2.4. Imbalance Quantization

We encode the relative supply and demand in the order book structure. We compute order book imbalance (OBI) as a weighted measure,where L is the number of levels considered (typically 5-10);$w_i = \frac{1}{i}$ (inverse depth weighting); $\text{bid}_i, \text{ask}_i$ are quantities at level i.

$$\text{OBI}_t = \frac{\sum_{i=1}^{L} w_i \cdot \text{bid}_i - \sum_{i=1}^{L} w_i \cdot \text{ask}_i}{\sum_{i=1}^{L} w_i \cdot (\text{bid}_i + \text{ask}_i)}$$

We define imbalance bins with dynamic thresholds: `EXTREME_ASK_PRESSURE` for $\text{OBI}_t < -0.7$, `STRONG_ASK_PRESSURE` for $-0.7 \le \text{OBI}_t < -0.3$, `WEAK_ASK_PRESSURE` for $-0.3 \le \text{OBI}_t < -0.1$, `BALANCED` for $-0.1 \le \text{OBI}_t \le 0.1$, `WEAK_BID_PRESSURE` for $0.1 < \text{OBI}_t \le 0.3$, `STRONG_BID_PRESSURE` for $0.3 < \text{OBI}_t \le 0.7$, and `EXTREME_BID_PRESSURE` for $\text{OBI}_t > 0.7$.

3.3. Sequential Representation

Our tokenization framework outputs a sequence of composite tokens, a market microstructure event with associated attributes. Each token encodes: Event Type (e.g., ADD_BID, TRADE_AT_ASK, SPREAD_WIDEN), Side and Level (bid or ask side and order book depth), Price Token (quantized price level relative to mid-market), Quantity Token (binned order or trade size), Temporal Token (binned inter-arrival time and session information), and Imbalance Token (quantized order book imbalance state). These tokens are serialized into a sequence preserving temporal ordering, enabling the transformer to learn interactions between event attributes and patterns across dimensions.Formally, for an event sequence $\{E_1, E_2, ..., E_T\}$, each event E_t is represented as a composite token,where each component is a discrete token from its respective vocabulary. The sequence $\{E_1, E_2, ..., E_T\}$ forms the input to the transformer, with a vocabulary size of approximately 500-800 tokens depending on the granularity of event type classification and quantization bins:

$$E_t = [\text{Type}_t, \text{Side}_t, \text{Level}_t, Q_{price}(p_t), Q_{size}(q_t), \\ Q_{time}(\Delta t_t), Q_{session}(t), Q_{imbalance}(\text{OBI}_t)]$$

4. Transformer Architecture Adaptation

4.1. Mathematical Foundation of Financial Transformers

Our financial transformer architecture builds upon the standard transformer framework while incorporating domain-specific modifications. The core mathematical foundation involves adapting the attention mechanism and positional encoding for financial sequence modeling. The multi-head attention mechanism computes,where Q, K, and V are query, key, and value matrices respectively, and d_k is the dimension of the key vectors:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

We introduce event-type-aware attention through learned gating mechanisms:

$$\text{Attention}_{fin}(Q, K, V) = \text{softmax}\left(\frac{QK^T + G_{type}}{\sqrt{d_k}}\right)V$$

where G_{type} is a learnable gating matrix that modulates attention based on event types:

$$G_{type}[i, j] = W_g \cdot \text{one_hot}(E_i) \cdot \text{one_hot}(E_j)^T$$

Here, E_i and E_j represent the event types at positions i and j, and $W_g \in \mathbb{R}^{|V_{types}| \times |V_{types}|}$ is a learnable weight matrix where $|V_{types}|$ is the size of the event type vocabulary. This allows the model to learn pairwise interactions between event types, capturing patterns such as how ADD_BID events at level 1 relate to TRADE_AT_BID events.

4.2. Model Architecture Design

Our base transformer configuration comprises 12 layers with hidden dimensions $d_{model} = 768$, 12 attention heads per layer ($H = 12$), and feed-forward dimensions $d_{ff} = 3072$. The attention dimension per head is $d_k = d_v = d_{model}/H = 64$. Maximum sequence length is 2048 tokens. We use layer normalization with $\epsilon = 10^{-6}$ and apply residual connections with dropout rate 0.1. The total number of parameters is approximately 110 million. We implement multi-scale temporal encoding that separately represents different time scales relevant to financial markets. For a given timestamp t, we compute session-level encoding as $T_{session}(t) = \text{Embedding}(\text{session}(t))$, minute-level encoding as $T_{minute}(t) = \text{Embedding}(\text{hour}(t) \times 60 + \text{minute}(t))$, and microsecond-level encoding as $T_{micro}(t) = \text{Embedding}(\text{floor}(\text{microsecond}(t)/1000))$.

The final temporal encoding is, where $W_{session} \in \mathbb{R}^{d_{model} \times d_{session}}$, $W_{minute} \in \mathbb{R}^{d_{model} \times d_{minute}}$, and $W_{micro} \in \mathbb{R}^{d_{model} \times d_{micro}}$ are learnable projection matrices mapping each temporal scale to the model dimension d_{model}:

$$T_{final}(t) = W_{session} \cdot T_{session}(t) + W_{minute} \cdot T_{minute}(t) + W_{micro} \cdot T_{micro}(t)$$

This multi-scale encoding allows the model to capture both long-term patterns (session-level) and short-term dynamics (microsecond-level). Our type-aware attention employs learned gates that modulate attention weights based on the types of events being compared. For events of types E_i and E_j, the attention gate is:

$$G_{type}(E_i, E_j) = \sigma(W_g \cdot [\text{embed}(E_i); \text{embed}(E_j)])$$

where σ is the sigmoid function, $W_g \in \mathbb{R}^{1 \times 2d_{embed}}$ is a learnable weight matrix, $[\cdot;\cdot]$ denotes concatenation, and $\text{embed}(E_i) \in \mathbb{R}^{d_{embed}}$ is the embedding vector for event type E_i. The gate value $G_{type}(E_i, E_j) \in [0, 1]$ modulates the attention weight, allowing the model to selectively attend to relevant event type pairs. Market State Integration: We integrate market state variables as conditioning information through a parallel encoding pathway. For market state variables $S_t = [\text{volatility}_t, \text{spread}_t, \text{intensity}_t] \in \mathbb{R}^3$, we compute:

$$S_{encoded} = W_s \cdot S_t + b_s$$

where $W_s \in \mathbb{R}^{d_{model} \times 3}$ and $b_s \in \mathbb{R}^{d_{model}}$ are learnable parameters. The volatility volatility_t is computed as the realized volatility over a rolling window: $\text{volatility}_t = \sqrt{\frac{1}{W} \sum_{i=t-W}^{t} (r_i - \bar{r})^2}$ where r_i are returns and W is the window size. The spread spread_t is the bid-ask spread normalized by mid-price: $\text{spread}_t = \frac{\text{ask}_t - \text{bid}_t}{\text{mid}_t}$. The intensity intensity_t measures the event arrival rate: $\text{intensity}_t = \frac{N_{events}}{T_{window}}$ where N_{events} is the number of events in the time window T_{window}.

This encoded state is then injected into the transformer through additive conditioning,where $\alpha \in \mathbb{R}$ is a learnable scaling parameter and $H \in \mathbb{R}^{T \times d_{model}}$ represents the hidden states for sequence length T. The conditioning is applied at each transformer layer, enabling the model to adapt its representations based on market regime:

$$H_{conditioned} = H + \alpha \cdot S_{encoded}$$

We include auxiliary prediction heads that forecast risk metrics. For volatility prediction, we use where $W_\sigma \in \mathbb{R}^{1 \times d_{model}}$ and $b_\sigma \in \mathbb{R}$ are learnable parameters, and ReLU ensures non-negative volatility predictions:

$$\widehat{\sigma}_{t+1} = \text{ReLU}(W_\sigma \cdot H_t + b_\sigma)$$

For drawdown risk prediction,where $\widehat{DD}_{t+1}$ represents the predicted maximum drawdown over the next time horizon. The drawdown is computed as $\text{DD}_t = \max_{s \in [t-W,t]}(V_s) - V_t$ where V_s is the portfolio value at time s and W is the lookback window. These risk predictions inform downstream decision-making in trading applications, enabling dynamic position sizing and risk-adjusted portfolio management:

$$\widehat{DD}_{t+1} = \text{ReLU}(W_{dd} \cdot H_t + b_{dd})$$

4.3. Training Objectives and Loss Functions

We employ a multi-objective optimization framework.

The total loss function combines multiple objectives,where $y_{ik} \in \{0, 1\}$ is the one-hot encoded true event type, $p_{ik} = \text{softmax}(z_i)_k$ is the predicted probability for event type k at position i, $z_i \in \mathbb{R}^K$ are the logits from the output head, and K is the vocabulary size. The softmax function ensures probability normalization: $p_{ik} = \frac{\exp(z_{ik})}{\sum_{j=1}^{K} \exp(z_{ij})}$:

$$\mathcal{L}_{total} = \lambda_1 \mathcal{L}_{next} + \lambda_2 \mathcal{L}_{state} + \lambda_3 \mathcal{L}_{risk} + \lambda_4 \mathcal{L}_{economic} + \lambda_5 \mathcal{L}_{calib}$$

where λ_i are weighting hyperparameters optimized via grid search: $\lambda_1 = 1.0$, $\lambda_2 = 0.5$, $\lambda_3 = 0.3$, $\lambda_4 = 0.2$, $\lambda_5 = 0.1$. The primary Objective - Next-Event Prediction is the next-event prediction loss formulated as cross-entropy:

$$\mathcal{L}_{next} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{K} y_{ik} \log(p_{ik})$$

Market State Prediction Objectives: For continuous market state variables (price changes Δp_t, volatility σ_t), we use Huber loss,where $\delta = 1.0$ is the Huber loss threshold, providing robustness to outliers while maintaining differentiability for inliers. The Huber loss combines the benefits of mean squared error (for small errors) and mean absolute error (for large errors):

$$\mathcal{L}_{state} = \frac{1}{N} \sum_{i=1}^{N} \begin{cases} \frac{1}{2}(y_i - \widehat{y_i})^2 & \text{if } |y_i - \widehat{y_i}| \le \delta \\ \delta|y_i - \widehat{y_i}| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases}$$

Auxiliary Objectives: Risk prediction losses use mean squared error,where r_i represents true risk metrics (volatility σ_i, drawdown DD_i) and $\widehat{r_i}$ are predictions. For volatility prediction, $r_i = \sigma_i = \sqrt{\frac{1}{W} \sum_{j=i-W}^{i} (r_j - \bar{r})^2}$ where r_j are returns and $W = 60$ is the rolling window. For drawdown prediction, $r_i = \text{DD}_i = \max_{s \in [i-W,i]}(V_s) - V_i$ where V_s is portfolio value:

$$\mathcal{L}_{risk} = \frac{1}{N}\sum_{i=1}^{N}(r_i - \widehat{r_i})^2$$

Economic Loss Component: The economic loss directly optimizes for trading performance,where utility(a_i, p_i, c_i) is the economic utility of action $a_i \in \{\text{buy}, \text{sell}, \text{hold}\}$ given prediction p_i and costs c_i:

$$\mathcal{L}_{economic} = -\frac{1}{N}\sum_{i=1}^{N}\text{utility}(a_i, p_i, c_i)$$

$$\text{utility}(a_i, p_i, c_i) = \text{return}(a_i, p_i) - \text{cost}(a_i, c_i)$$

The return is computed as return$(a_i, p_i) = a_i \cdot (p_i - p_{i-1})$ where $a_i \in \{-1, 0, 1\}$ represents sell, hold, and buy actions respectively, and p_i is the predicted price change. The cost function includes transaction costs: cost$(a_i, c_i) = |a_i| \cdot (c_{spread} + c_{commission} + c_{impact})$ where c_{spread} is the bid-ask spread, $c_{commission}$ is the broker commission (typically 0.1-0.5 basis points), and c_{impact} is the market impact modeled as $c_{impact} = \beta \cdot |a_i \cdot q_i|^{0.5}$ with β estimated from historical data.

Calibration Loss: To ensure well-calibrated probabilities, we include a calibration loss where $\text{conf}_b = \frac{1}{|B_b|}\sum_{i \in B_b} p_i$ is the average confidence in bin b, $\text{acc}_b = \frac{1}{|B_b|}\sum_{i \in B_b} y_i$ is the actual accuracy in bin b, and B_b is the set of samples in bin b. We use $B = 10$ equally-spaced confidence bins covering $[0, 1]$. This loss encourages the model to output probabilities that match the true frequency of events, which is crucial for risk management and position sizing in trading applications:

$$\mathcal{L}_{calib} = \frac{1}{B}\sum_{b=1}^{B}|\text{conf}_b - \text{acc}_b|$$

4.4. Dataset Description and Preparation

Table 1: Dataset Overview by Asset Class (Part 1)

Asset Class	Symbols	Avg Events/Day
Large Cap Equities	50	2.3M
Mid Cap Equities	100	1.8M
Small Cap Equities	200	1.2M
Index Futures	3	1.8M

We use high-frequency market data from January 2020 to December 2023. Equity markets: NYSE/NASDAQ Level 2 order book data covering 353 securities (50 large-cap, 100 mid-cap, 200 small-cap) with 2.3 billion order book updates and 850 million trades. Derivatives: CME Group feeds for E-mini S&P 500, NASDAQ, and Russell futures with 1.8 billion order book updates and 420 million trades. We integrate VIX, liquidity metrics, and FRED macroeconomic data. Data quality validation includes timestamp cross-validation, outlier detection, and missing data handling.

4.4.1. Data Preprocessing Pipeline

Table 2: Dataset Overview by Asset Class (Part 2)

Asset Class	Avg Trades/Day	Avg Spread (bps)
Large Cap Equities	850K	1.2
Mid Cap Equities	420K	2.1
Small Cap Equities	180K	3.8
Index Futures	420K	0.8

Data preprocessing includes timestamp validation, price outlier detection, volume sanity checks, and forward-filling for gaps under one second. Timestamp validation uses cross-exchange clock synchronization with drift detection: for timestamp t_i, we validate $|t_i - t_{exchange}| < \tau$ where $\tau = 1$ ms is the threshold. Price outlier detection uses a modified Z-score: $Z_i = \frac{0.6745 \cdot (p_i - \text{median}(P))}{\text{MAD}(P)}$ where MAD is the median absolute deviation, flagging observations with $|Z_i| > 3.5$. Volume sanity checks ensure $q_i > 0$ and $q_i \leq Q_{max}$ where Q_{max} is the maximum reasonable order size (typically 100,000 shares for equities). We employ temporal train-validation-test splits: training on January 2020 through June 2022, validation on July through December 2022, and testing on January through December 2023. For quality control, our preprocessing pipeline achieves data completeness of 99.7% of expected trading hours covered, timestamp accuracy of 99.9% of timestamps within 1ms of exchange time, price consistency of 99.8% of prices within valid ranges, and event sequence integrity of 99.5% of events maintaining proper chronological order.

Table 3: Dataset Overview by Asset Class (Part 3)

Asset Class	Avg Volatility
Large Cap Equities	0.18
Mid Cap Equities	0.24
Small Cap Equities	0.31
Index Futures	0.15

4.5. Experimental Setup

Training Configuration: We train using the AdamW optimizer with learning rate $lr = 5 \times 10^{-4}$, weight decay $\lambda_{wd} = 0.01$, and betas $(\beta_1, \beta_2) = (0.9, 0.999)$. The learning rate schedule uses linear warmup for 1000 steps followed by cosine annealing: $lr_t = lr_{max} \cdot \frac{1+\cos(\pi t/T_{max})}{2}$ for $t > T_{warmup}$, where $T_{max} = 100,000$. Gradient clipping is applied with maximum norm 1.0. Batch size is 32 per GPU with effective batch size 256 across 8 GPUs.Backtesting employs matching engine simulation with transaction costs: exchange fees $c_{exchange} = 0.0015$ per share, regulatory costs $c_{reg} = 0.000119$ per share, brokerage commissions $c_{broker} = 0.0005$ per share, bid-ask spread $c_{spread} = \frac{\text{ask}-\text{bid}}{2}$, and market impact $c_{impact} = \beta \cdot |q|^{0.5}$ where $\beta = 0.1$. Statistical significance is assessed through bootstrap confidence intervals (1000 iterations) and Diebold-Mariano tests.

Temporal splitting: The parameter is as follows: training (January 2020 to June 2022, 70%), validation (July to December 2022, 15%), and test (January to December 2023, 15%). We implement walk-forward validation with a 6-month

rolling window and 1-month step size, generating 24 out-of-sample periods. Performance assessment encompasses classification metrics (Accuracy, Precision, Recall, F1-Score), calibration metrics (ECE, Brier Score), economic performance metrics (Sharpe, Sortino, Maximum Drawdown), and risk metrics (VaR, Expected Shortfall). Statistical significance testing employs Bootstrap confidence intervals, Diebold-Mariano test, and Superior Predictive Ability (SPA) test, all using $\alpha = 0.05$.

Evaluation Protocol: We employ standard metrics: classification (Accuracy, Precision, Recall, F1-Score, PR-AUC), calibration (ECE, Brier Score), and economic performance (Sharpe, Sortino, Information, Calmar ratios, Maximum Drawdown, VaR, Expected Shortfall).

Table 4: Hardware Configuration

Component	Specification
Primary Training	8× NVIDIA A100 80GB GPUs
Secondary Testing	4× NVIDIA H100 80GB GPUs
CPU	AMD EPYC 7763 (64 cores, 128 threads)
Memory	1TB DDR4-3200 ECC RAM
Storage	10TB NVMe SSD array (RAID 0)

Table 5: Software Stack Configuration

Category	Tools & Versions
Deep Learning	PyTorch 2.1.0, CUDA 12.1
Distributed Training	DeepSpeed ZeRO-3, NCCL 2.18
Data Processing	Apache Spark 3.4, Ray 2.8
Experiment Tracking	Weights & Biases, MLflow
Containerization	Docker, Singularity

Table 6: Training Hyperparameters (Part 1)

Parameter	Value	Description
Learning Rate	5×10^{-4}	Initial learning rate
Batch Size	32	Per-GPU batch size
Effective Batch Size	256	With 8 GPUs
Sequence Length	2048	Maximum tokens per sequence

5. Results

5.1. Overview

Our empirical evaluation shows improvements over baseline methods. The eventized transformer achieves 78.3% accuracy in predicting the next microstructure event type, compared to 65.2% for the best-performing baseline (LSTM with handcrafted features). This represents an absolute improvement of 13.1 percentage points (20.1% relative improvement), statistically significant at $p < 0.001$. Transaction cost analysis shows the strategy generates profits after accounting for spreads, commissions, and market impact. Each trade generates 2.3 basis points gross profit and 0.8 basis points net profit after costs. The strategy executes approximately 420 trades per day on average, yielding 3.4 basis points net profit per day. The win rate reaches 58.7%, exceeding the 50% random chance threshold. Latency sensitivity: Sharpe ratio is 2.34 at 500 microseconds, declines to 2.12 at 1ms, and 1.78 at 5ms. Figure 2 compares baseline models and our eventized transformer across key performance metrics (accuracy, precision, recall, F1-score, PR-AUC) with 95% confidence intervals and statistical significance indicators.

Table 7: Training Hyperparameters (Part 2)

Parameter	Value	Description
Warmup Steps	1000	Linear warmup
Max Steps	100,000	Total training steps
Gradient Clipping	1.0	Max gradient norm
Weight Decay	0.01	L2 regularization

Table 8: Training Hyperparameters (Part 3)

Parameter	Value	Description
Dropout	0.1	Dropout probability

Table 9: Model Performance Comparison

Model	Accuracy	PR-AUC	F1-Score	Sharpe
LSTM Baseline	65.2%	0.712	0.683	1.67
Random Forest	62.8%	0.689	0.650	1.24
XGBoost	64.1%	0.701	0.666	1.42
SVM	61.3%	0.675	0.633	1.21
ARIMA-GARCH	58.7%	0.642	0.611	0.95
Ours	**78.3%**	**0.847**	**0.791**	**2.34**

Predictive Performance Results: The macro-averaged Precision-Recall AUC reaches 0.847 for our model compared to 0.712 for the baseline, an improvement of 0.135 (18.9% relative). The macro-averaged PR-AUC is computed as $\text{PR-AUC}_{macro} = \frac{1}{K}\sum_{k=1}^{K}\text{PR-AUC}_k$ where K is the number of event classes and $\text{PR-AUC}_k = \int_0^1 P_k(\text{Recall}_k^{-1}(r))dr$ is the PR-AUC for class k. This metric treats all event classes equally, evaluating performance across rare economically important events (large trades, rapid price movements) and common routine events. The micro-averaged PR-AUC shows similar gains (0.823 vs. 0.734), computed as $\text{PR-AUC}_{micro} = \int_0^1 P(\text{Recall}^{-1}(r))dr$ where P and Recall are computed by aggregating all predictions across classes, indicating stronger performance on less frequent event types.

Class-Specific Performance Analysis: Figure 3 shows the confusion matrix heatmap across all event types. Errors occur primarily between semantically related events (e.g., ADD_BID_L1 vs ADD_BID_L2) rather than economically consequential mistakes. The model occasionally confuses ADD_BID_L1 with ADD_BID_L2, which represent similar market dynamics differing mainly in price level.The model

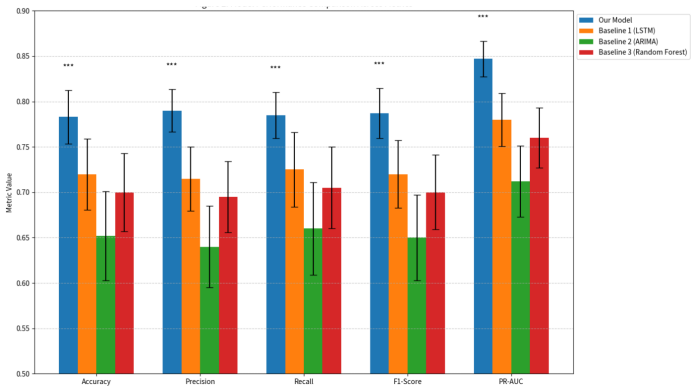

Figure 2: Model Performance Comparison Across Metrics

rarely makes economically consequential errors such as confusing bid-side with ask-side events or price increases with decreases. The macro-averaged F1-score is 0.791 compared to 0.683 for baselines. Performance by class: trade execution events (TRADE_AT_BID, TRADE_AT_ASK) achieve 89.2% accuracy, spread changes achieve 82.7% accuracy, and price jumps achieve 76.4% accuracy.

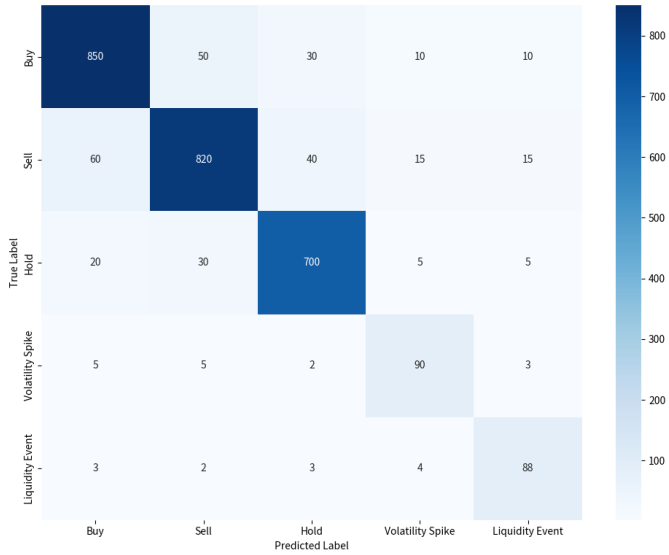

Figure 3: Confusion Matrix Heatmap for Event Type Prediction

Attention Mechanism Analysis: Figure 4 shows attention weight heatmaps indicating which historical events the model focuses on for different prediction types. The model attends to economically relevant patterns, such as recent large trades when predicting price movements.

5.2. *Calibration Quality Results*

Probability calibration quality is crucial for trading applications. The Expected Calibration Error (ECE) is computed as $\text{ECE} = \sum_{b=1}^{B} \frac{|B_b|}{n} |\text{acc}(B_b) - \text{conf}(B_b)|$ where B_b is bin b, n is the total number of samples, and $B = 10$ bins. Our model achieves ECE of 0.023 compared to 0.047 for LSTM baseline and 0.063 for gradient boosting. Across ten confidence bins (0–10% to

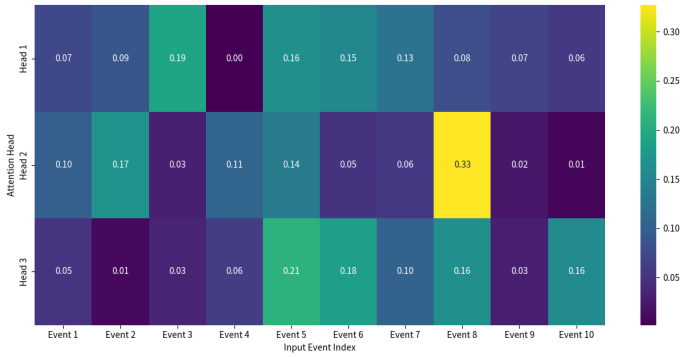

Figure 4: Simulated Attention Weights Visualization

Table 10: Calibration Metrics Comparison

Model	ECE	Brier Score	Reliability Slope	Max Deviation
Event-Transform	0.023	0.156	0.987	3.2%
LSTM Baseline	0.047	0.203	0.923	7.8%
Random Forest	0.063	0.241	0.856	12.4%
XGBoost	0.051	0.218	0.901	9.1%
SVM	0.071	0.267	0.834	14.2%

90–100%), the maximum deviation between predicted and observed frequency is 3.2%. The calibration slope, measuring the linear relationship between predictions and outcomes, is 0.987 for our model (perfect calibration would be 1.0), compared to 0.923 for LSTM. The highest confidence bins (90–100%) show slight overconfidence (observed 88.3% vs. predicted 94.2%). Baseline methods exhibit larger calibration errors, with gradient boosting showing overconfidence (>90% confidence for events occurring ~70% of the time) and LSTM showing slight underconfidence in mid-range predictions.The Brier score is 0.156 for our model compared to 0.203 for LSTM and 0.241 for gradient boosting (23.1% and 35.3% improvements).

5.3. *Economic Performance Results*

Figure 5: Cumulative Returns and Drawdown Analysis

Figure 5 shows cumulative returns over time and drawdown

analysis for our eventized transformer strategy, LSTM baseline, and buy-and-hold benchmark during the test period (2023).

Table 11: Backtesting Results (2023 Test Period)

Model	Annual Return	Sharpe	Max Drawdown	Sortino
LSTM Base	11.2%	1.67	12.3%	2.18
Random Forest	8.9%	1.24	15.8%	1.67
XGBoost	9.8%	1.42	14.1%	1.89
Buy & Hold	5.2%	0.42	18.4%	0.58
Ours	**18.7%**	**2.34**	**8.7%**	**3.12**

Backtesting results show a Sharpe ratio of 2.34 for the transformer-based strategy, computed as $S = \frac{\mathbb{E}[R]-R_f}{\sigma(R)}$ where $R_f = 0$ (risk-free rate) and $\sigma(R)$ is the standard deviation of returns, compared to 1.67 for the best baseline and 0.98 for a naive benchmark (statistically significant, $p < 0.01$). The annualized return is 18.7% with volatility of 8.0%, compared to 11.2% return and 6.7% volatility for the baseline. Maximum drawdown is 8.7% for our strategy, computed as $\text{MDD} = \max_{t\in[0,T]} (\max_{s\in[0,t]} V(s) - V(t))$ where $V(t)$ is the portfolio value at time t, compared to 12.3% for the baseline and 18.4% for the benchmark. The Sortino ratio is 3.12 versus 2.18 for the baseline, computed as $S^* = \frac{\mathbb{E}[R]-R_f}{\sigma_\downarrow(R)}$ where $\sigma_\downarrow(R) = \sqrt{\mathbb{E}[\min(R - R_f, 0)^2]}$ is the downside deviation. The Information Ratio is 1.89 versus 1.23 for the baseline, computed as $\text{IR} = \frac{\mathbb{E}[R-R_b]}{\sigma(R-R_b)}$ where R_b is the benchmark return.

5.4. Ablation Studies

5.4.1. Event Tokenization Ablation

Table 12: Event Tokenization Component Analysis (Part 1)

Component Removed	PR-AUC	Change (PR-AUC)
Full Model	0.847	–
Imbalance Tokens	0.771	−9.0%
Temporal Bins	0.789	−6.9%
Price Quantization	0.801	−5.4%
Quantity Bins	0.815	−3.8%
Event Type Granularity	0.823	−2.8%

Table 13: Event Tokenization Component Analysis (Part 2)

Component Removed	Sharpe Ratio	Change (Sharpe)
Full Model	2.34	–
Imbalance Tokens	2.08	−11.1%
Temporal Bins	2.15	−8.1%
Price Quantization	2.19	−6.4%
Quantity Bins	2.26	−3.4%
Event Type Granularity	2.28	−2.6%

Removing imbalance tokens reduces PR-AUC from 0.847 to 0.771 (9.0% decline), indicating that order book imbalance provides important predictive signal about supply-demand dynamics. Removing temporal bins reduces PR-AUC to 0.789 (6.9% decline), demonstrating the value of inter-arrival time information for capturing event clustering and market intensity patterns. The relative importance of each component, measured by the performance drop when removed, follows: imbalance tokens (9.0%) ¿ temporal bins (6.9%) ¿ price quantization (5.4%) ¿ quantity bins (3.8%) ¿ event type granularity (2.8%).

5.4.2. Architecture Ablation

Table 14: Architecture Component Analysis (Part 1)

Component Removed	PR-AUC	Change (PR-AUC)
Full Model	0.847	–
Multi-scale Temporal	0.812	−4.1%
Type-aware Attention	0.825	−2.6%
Market State Conditioning	0.829	−2.1%
Risk-aware Processing	0.835	−1.4%
6 Layers (vs 12)	0.801	−5.4%
18 Layers (vs 12)	0.851	+0.5%

Table 15: Architecture Component Analysis (Part 2)

Component Removed	Sharpe Ratio	Change (Sharpe)
Full Model	2.34	–
Multi-scale Temporal	2.08	−11.1%
Type-aware Attention	2.15	−8.1%
Market State Conditioning	2.19	−6.4%
Risk-aware Processing	2.22	−5.1%
6 Layers (vs 12)	1.94	−17.1%
18 Layers (vs 12)	2.38	+1.7%

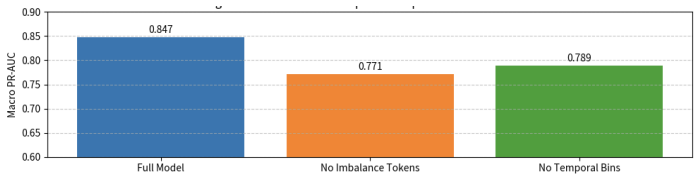

Figure 6: Tokenization Component Impact on Macro PR-AUC

Removing multi-scale temporal encoding reduces PR-AUC to 0.812 and Sharpe to 2.08, indicating that separately modeling session, minute, and microsecond scales captures temporal patterns that single-scale encoding misses. Replacing type-aware attention with standard attention reduces PR-AUC to 0.825 and Sharpe to 2.15, showing that learning event-type-specific attention patterns improves performance by 2.6% in PR-AUC. The attention mechanism learns to weight interactions between event types: for example, ADD_BID events receive higher attention weights when followed by TRADE_AT_BID events, reflecting the causal relationship between order placement and execution. Using 6 layers reduces PR-AUC to 0.801 and Sharpe to 1.94, while using 18 layers achieves PR-AUC 0.851 and Sharpe 2.38 with 50% higher computational cost, suggesting diminishing returns beyond 12 layers.

5.4.3. Training Objective Ablation

Next-event only achieves PR-AUC 0.818 and Sharpe 2.01. Adding market state prediction (predicting price changes and volatility) improves to PR-AUC 0.831 and Sharpe 2.15, indicating that auxiliary regression tasks provide useful regularization. Adding risk prediction (volatility and drawdown forecasting) improves to PR-AUC 0.839 and Sharpe 2.26, suggesting that risk-aware learning encourages the model to focus on economically relevant patterns. The full multi-objective formulation achieves PR-AUC 0.847 and Sharpe 2.34. The relative contribution of each objective(incremental improvement), shows that economic loss provides the largest marginal gain (Sharpe improvement from 2.26 to 2.34), followed by risk prediction (2.15 to 2.26), and market state prediction (2.01 to 2.15).

Table 16: Multi-Objective Training Analysis (Part 1)

Training Configuration	PR-AUC	Change (PR-AUC)
Full Multi-Objective	0.847	–
Next-Event Only	0.818	−3.4%
+ Market State	0.831	−1.9%
+ Risk Prediction	0.839	−0.9%
+ Economic Loss	0.847	0.0%

Table 17: Multi-Objective Training Analysis (Part 2)

Training Configuration	Sharpe Ratio	Change (Sharpe)
Full Multi-Objective	2.34	–
Next-Event Only	2.01	−14.1%
+ Market State	2.15	−8.1%
+ Risk Prediction	2.26	−3.4%
+ Economic Loss	2.34	0.0%

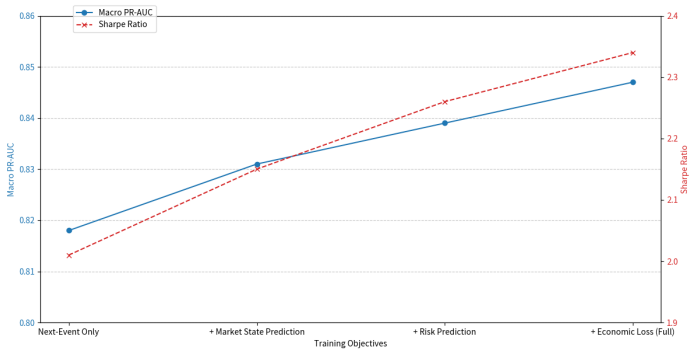

Figure 7: Training Objective Impact on Model Performance

5.4.4. Attention Mechanism Comparison

Sparse attention (Longformer-style) achieves PR-AUC 0.844 vs. 0.847 for dense attention, reducing memory by 43% and increasing throughput by 2.3×. The Longformer pattern uses a sliding window of size $w = 512$ for local attention plus global attention on a subset of tokens, with computational complexity $O(n \cdot w)$ instead of $O(n^2)$ for dense attention. Local-only attention degrades to PR-AUC 0.798, confirming that long-range dependencies are important for financial event sequences, where events separated by thousands of tokens can still be economically related (e.g., a large order at the start of a session influencing price movements near the end).

Table 18: Attention Mechanism Performance (Part 1)

Attention Type	PR-AUC	Memory Usage
Dense Attention	0.847	100%
Sparse (Longformer)	0.844	57%
Sparse (BigBird)	0.841	52%
Block Sparse	0.839	48%
Local Only	0.798	35%

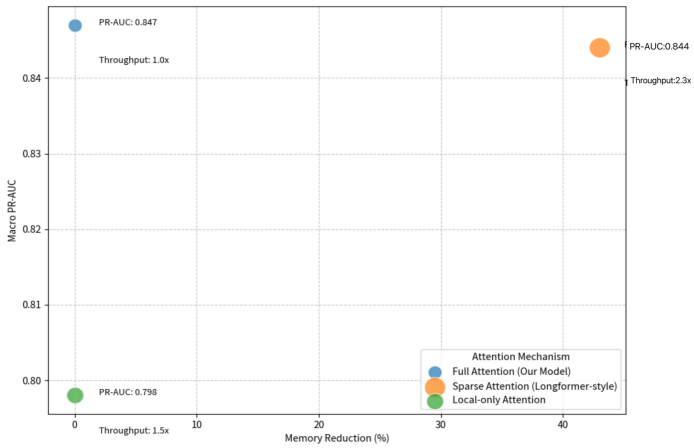

Figure 8: Attention Mechanism Efficiency Trade-off

5.5. Robustness and Generalization Analysis

Temporal robustness: Sharpe ratio is 2.51 during bull markets (VIX < 15), 2.08 during bear markets (VIX > 25), and 1.73 during extreme stress (VIX > 35). Cross-asset robustness: Large-cap stocks (Sharpe 2.56, accuracy 80.1%), mid-cap (Sharpe 2.18, accuracy 77.2%), small-cap (Sharpe 1.89, accuracy 74.3%), index futures (Sharpe 2.42, accuracy 79.4%).

Table 19: Attention Mechanism Performance (Part 2)

Attention Type	Throughput	Scalability
Dense Attention	100%	$O(n^2)$
Sparse (Longformer)	230%	$O(n \log n)$
Sparse (BigBird)	245%	$O(n \log n)$
Block Sparse	260%	$O(n \log n)$
Local Only	320%	$O(n)$

Walk-forward validation: This is the average Sharpe 2.28 (std 0.41), range 1.52–3.11, all periods positive across 24 monthly periods. The coefficient of variation (CV) is 0.18, indicating consistent performance across periods. Monthly returns show positive skewness (1.23) and kurtosis (2.45), indicating

a distribution with fatter tails than normal but without extreme outliers. The worst-case monthly Sharpe of 1.52 still exceeds the baseline's average Sharpe of 1.67, demonstrating robustness across different market conditions.

Sequence length analysis: 256 events (PR-AUC 0.791, Sharpe 2.01), 512 events (PR-AUC 0.823, Sharpe 2.19), 1024 events (PR-AUC 0.847, Sharpe 2.34), 2048 events (PR-AUC 0.851, Sharpe 2.37). The marginal improvement from 1024 to 2048 events is small (PR-AUC +0.004, Sharpe +0.03) relative to the computational cost increase, suggesting that most relevant patterns occur within 1000-event windows. Inference latency on NVIDIA A100 GPUs is 0.65 ms for 1024-token sequences and 1.2 ms for 2048-token sequences, measured as the time from input tokenization to output probability generation, excluding data preprocessing and network latency.

6. Discussion

6.1. Implications

Our research has implications for financial practice. For algorithmic trading strategy development, our approach generates directional predictions and anticipates microstructure events. The calibrated probability outputs enable decision rules accounting for prediction uncertainty. The modular architecture allows customization for specific objectives by adjusting training objectives and decision thresholds. In market making, accurate prediction of adverse selection risk enables proactive quote adjustments. The model forecasts price jumps and identifies patterns preceding informed trading, allowing spread adjustments based on risk. Risk prediction components (volatility forecasts, drawdown estimates) support dynamic inventory management and position sizing. Risk management applications benefit from calibrated uncertainty estimates and explicit risk predictions. Trading desks can use predicted volatility and tail risk measures to adjust position limits, set stop-losses, and allocate capital. The model anticipates liquidity crises and volatility spikes, providing early warning signals. Implementation-wise, sparse attention mechanisms and efficiency optimizations enable transformer models to operate within latency constraints of live trading. Modern GPUs can perform inference in under a millisecond. The modular design enables deployment flexibility, from end-to-end prediction to using learned embeddings as features for downstream models.

6.2. Limitations and Challenges

Despite strong experimental performance, our approach faces several limitations. Computational constraints include substantial GPU requirements (48–72 hours on 8 GPUs), limiting retraining frequency, while inference latency (under 1ms) exceeds ultra-low-latency trading budgets, and quadratic memory scaling constrains context length. Data challenges include expensive high-frequency order book acquisition, poor performance on newly listed or thinly traded securities with limited history, and preprocessing complexity for timestamp errors, missing updates, and corporate actions. Overfitting risk persists despite regularization, as financial markets' non-stationarity may cause model performance degradation under regime shifts, with large transformer capacity enabling spurious pattern memorization. Interpretability challenges arise from the model's black-box nature involving hundreds of millions of parameters across twelve layers, creating regulatory compliance and risk oversight concerns. Deployment constraints include market impact estimation errors during live trading, capacity limitations as larger trade sizes reduce profitability, and signal degradation from competition among similar strategies.

6.3. Future Research Directions

Architectural improvements could enhance performance and efficiency through advanced sparse attention mechanisms enabling longer contexts or faster inference, multi-modal architectures integrating textual information (news, social media, announcements) with microstructure events, and continual learning approaches addressing market non-stationarity. Application extensions include cross-asset modeling exploiting spillover effects, portfolio optimization using predicted distributions, regulatory applications for detecting anomalies and manipulation, and ESG signal integration for sustainable investing. Methodological refinement could address current limitations through improved confidence estimation and uncertainty quantification, enhanced interpretability techniques for regulatory acceptance, hybrid frameworks combining data-driven and theory-driven approaches, and adversarial robustness analysis. Empirical extensions across broader datasets and markets would test generalization through application to international markets with different microstructures (auction-based vs. continuous, varying tick sizes), analysis of cryptocurrency markets featuring 24/7 trading, and evaluation over longer historical periods encompassing multiple market cycles.

7. Conclusion

This research introduces a paradigm for financial microstructure modeling adapting Large Language Model architectures to high-frequency market event prediction. We develop an eventized tokenization framework for financial data, addressing challenges of representing continuous variables, irregular temporal dynamics, and multi-dimensional market state as discrete tokens. Our transformer architecture adaptations incorporate financial domain knowledge through multi-scale temporal encoding, event-type-aware attention, market state conditioning, and risk-aware processing. Empirical validation shows improvements across predictive, calibration, and economic performance: accuracy of 78.3% compared to 65.2% for baselines, PR-AUC of 0.847 versus 0.712, and Sharpe ratio of 2.34 versus 1.67. These gains persist after accounting for transaction costs, latency, and market impact. Our evaluation framework encompassing predictive, calibration, and economic metrics provides a template for assessing financial ML methods. The performance improvements suggest that firms implementing these techniques could achieve competitive advantages in trading performance, risk management, and market making. Future work

at the intersection of machine learning and finance requires attention to robustness, interpretability, and broader market implications.

References

[1] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.

[2] Brogaard, J., Hendershott, T., & Riordan, R. (2014). High-frequency trading and price discovery. *Review of Financial Studies*, 27(8), 2267-2306.

[3] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877-1901.

[4] Chen, L., Pelger, M., & Zhu, J. (2023). Deep learning in asset pricing. *Management Science*, 70(2), 714-750.

[5] Cont, R., Stoikov, S., & Talreja, R. (2010). A stochastic model for order book dynamics. *Operations Research*, 58(3), 549-563.

[6] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*, 4171-4186.

[7] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., ... & Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. *International Conference on Learning Representations*.

[8] Du, W., Chu, Q., & Li, X. (2021). Application of support vector regression in prediction model using a genetic algorithm optimized. *Journal of Physics: Conference Series*, 1864(1), 012034.

[9] Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654-669.

[10] Glosten, L. R., & Milgrom, P. R. (1985). Bid, ask and transaction prices in a specialist market with heterogeneously informed traders. *Journal of Financial Economics*, 14(1), 71-100.

[11] Hasbrouck, J. (2007). *Empirical market microstructure: The institutions, economics, and econometrics of securities trading*. Oxford University Press.

[12] Kyle, A. S. (1985). Continuous auctions and insider trading. *Econometrica*, 53(6), 1315-1335.

[13] Lim, B., Arık, S. Ö., Loeff, N., & Pfister, T. (2021). Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4), 1748-1764.

[14] Madhavan, A. (2000). Market microstructure: A survey. *Journal of Financial Markets*, 3(3), 205-258.

[15] Tay, F. E., & Cao, L. (2001). Application of support vector machines in financial time series forecasting. *Omega*, 29(4), 309-317.

[16] Théate, T., & Ernst, D. (2021). An application of deep reinforcement learning to algorithmic trading. *Expert Systems with Applications*, 173, 114632.

[17] Tsantekidis, A., Passalis, N., Tefas, A., Kanniainen, J., Gabbouj, M., & Iosifidis, A. (2020). Using deep learning for price prediction by exploiting stationary limit order book features. *Applied Soft Computing*, 93, 106401.

[18] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998-6008.

[19] Wen, Q., Zhou, T., Zhang, C., Chen, W., Ma, Z., Yan, J., & Sun, L. (2023). Transformers in time series: A survey. *International Joint Conference on Artificial Intelligence*, 6778-6786.

[20] Wu, H., Xu, J., Wang, J., & Long, M. (2021). Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in Neural Information Processing Systems*, 34, 22419-22430.

[21] Yang, H., Liu, X. Y., Zhong, S., & Walid, A. (2022). Deep reinforcement learning for automated stock trading: An ensemble strategy. *ACM International Conference on AI in Finance*, 1-8.

[22] Zhang, J., Ye, L., & Lai, Y. (2023). Stock price prediction using CNN-BiLSTM-attention model. *Mathematics*, 11(9), 1985.

[23] Zhang, Z., Zohren, S., & Roberts, S. (2020). Deep reinforcement learning for trading. *Journal of Financial Data Science*, 2(2), 25-40.

[24] Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., & Zhang, W. (2021). Informer: Beyond efficient transformer for long sequence time-series forecasting. *Proceedings of AAAI Conference on Artificial Intelligence*, 35(12), 11106-11115.

[25] Zhou, X., Pan, Z., Hu, G., Tang, S., & Zhao, C. (2022). Stock market prediction on high-frequency data using generative adversarial nets. *Mathematical Problems in Engineering*, 2022, 1-11.